第 5 课：梯度下降

吴恩达《Machine Learning Specialization》· Course 1 · Week 1 · Lesson 4

承上：第 4 课我们知道了——**训练模型 = 找到让成本函数 `J(w, b)` 最小的那对 (w, b)**。 **这一课**：请出真正"会发现答案"的算法——**梯度下降（Gradient Descent）**。它就是教机器怎么自动走到谷底。

本课主角算法：

$$w := w - \alpha \frac{\partial}{\partial w} J(w, b)$$

数学基础不好？ 这一课要用到"导数/斜率"，我在旁边的 **《数学补丁：导数与梯度》** 里从零讲了一遍，卡住就翻过去。

怎么用：从头往下读，灰色代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

运行下面这格，确认画图工具就绪、中文字体也设好。

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
# ---- 选择你电脑上存在的中文字体：Windows 用微软雅黑，Mac 用苹方，Linux 用 Noto ----
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、一个直觉：蒙眼下山

想象你**站在一座山上**，四周一片漆黑，你看不见谷底在哪。你手里只有一样东西——能**感知脚底下这一点点的坡度**。

你会怎么做？很简单：

1. 用脚感觉一下，**哪个方向是下坡**；
2. 朝着下坡方向，**迈一小步**；
3. 到了新位置，**再感觉一次坡度，再迈一步**；
4. 一直重复，直到走到一个**踩哪儿都不再往下走**的地方——那就是谷底。

梯度下降就是这个过程，只不过"山"是成本函数 `J`，"下坡"由导数（斜率）告诉我们。

- 你当前站在某个 `w` 上；
- 看看 `J` 在这点的**斜率**：往左是下坡还是往右是下坡？
- 往下坡方向挪一点点 `w`；
- 重复。

二、先只看一个参数：J(w) 的情形

为了好懂，我们先**固定 b = 0**，只看一个旋钮 `w`。成本函数就是一条 U 形曲线 `J(w)` ——还记得第 4 课那张"山谷"图吧。

梯度下降的算法（一个参数版）：

$$w := w - \alpha \frac{d}{dw} J(w)$$

`:=` 的意思是"**把 w 更新成右边这个新值**"。拆开看右边两样东西：

符号	名字	它在干嘛
`d/dw J(w)`	导数（就是那点的**斜率**）	告诉我们在这一点，往哪边是下坡
`α`（alpha）	**学习率**（learning rate）	决定"每一步迈多大"

为什么是"减"？关键就一个字——减完自然会朝下坡走：

- 如果斜率**是正的**（曲线向上翘）：`w - α×(正数)` → w **变小** → 往左挪（左边在下坡）
- 如果斜率**是负的**（曲线向下走）：`w - α×(负数)` → w **变大** → 往右挪（右边在下坡）

不管哪种情况，**w 都会自动朝谷底挪**。这就是"减号"的妙处。

三、亲手跑一遍梯度下降

光说不够，让它真的走给你看。用一个简单的成本函数 `J(w) = (w − 3)²`（谷底在 w = 3），从 w = 0 出发：

In [2]:

```
# 成本函数 J(w) = (w-3)^2，谷底在 w=3
def J(w):
    return (w - 3)**2

def dJ(w):                  # J 对 w 的导数（斜率）
    return 2 * (w - 3)

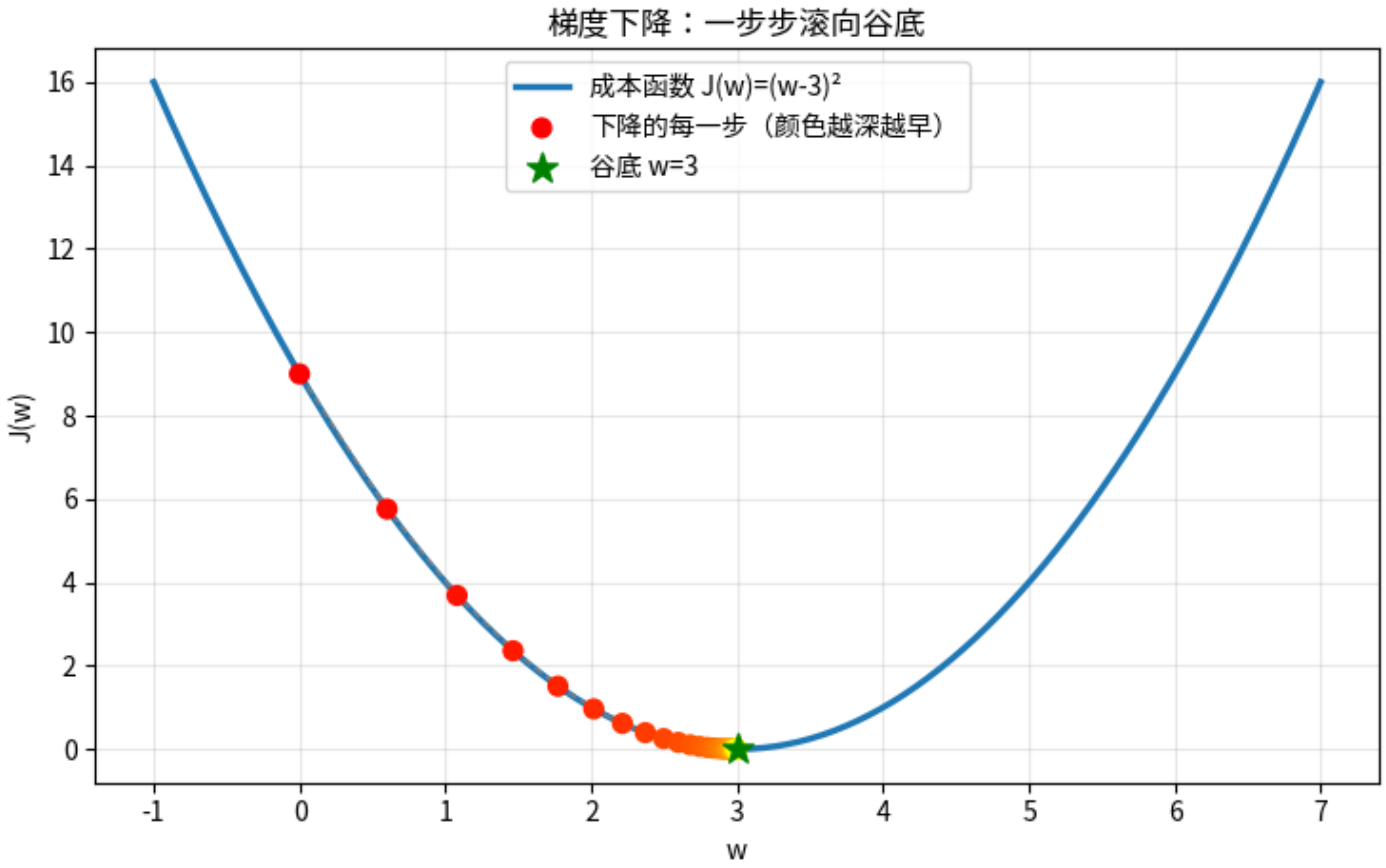
w = 0.0                     # 起点：站在 w=0
alpha = 0.1                 # 学习率：每步迈多大
path = [w]
for step in range(30):
    w = w - alpha * dJ(w)  # ← 梯度下降的核心，就这一行！
    path.append(w)

print("走了 30 步之后：")
print("  最终 w =", round(w, 4), "（谷底在 3）")
print("  此时的 J =", round(J(w), 8))
```

```
走了 30 步之后：
 最终 w = 2.9963 （谷底在 3）
 此时的 J = 1.379e-05
```

In [3]:

```
# 把下降的过程画出来
ws = np.linspace(-1, 7, 200)
plt.figure(figsize=(7.5, 4.8))
plt.plot(ws, J(ws), color="tab:blue", linewidth=2.4, label="成本函数 J(w)=(w-3)²")
p = np.array(path)
plt.scatter(p, J(p), c=range(len(p)), cmap="autumn", s=50, zorder=3,
            label="下降的每一步（颜色越深越早）")
plt.plot(p, J(p), "--", color="gray", linewidth=1, zorder=2)
plt.scatter([3], [0], color="green", s=140, marker="*", zorder=4, label="谷底 w=3")
plt.xlabel("w"); plt.ylabel("J(w)")
plt.title("梯度下降：一步步滚向谷底")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



看：小圆点从左上方一路"滚"向谷底，最后停在 `w = 3`、`J = 0`。这就是梯度下降干的事——**不用人告诉它答案，它自己沿坡度走到最优。**

四、学习率 α：步子迈多大，很关键

`α` 是这一课最容易"出事"的地方。看图就明白：

- `α` **太小** → 每步挪一点点，**慢吞吞**，要走很多步才到谷底；
- `α` **太大** → 一步迈过谷底，甚至**越走越高、直接飞出去**（这叫"发散"）；
- `α` **合适** → 又快又稳地到达谷底。

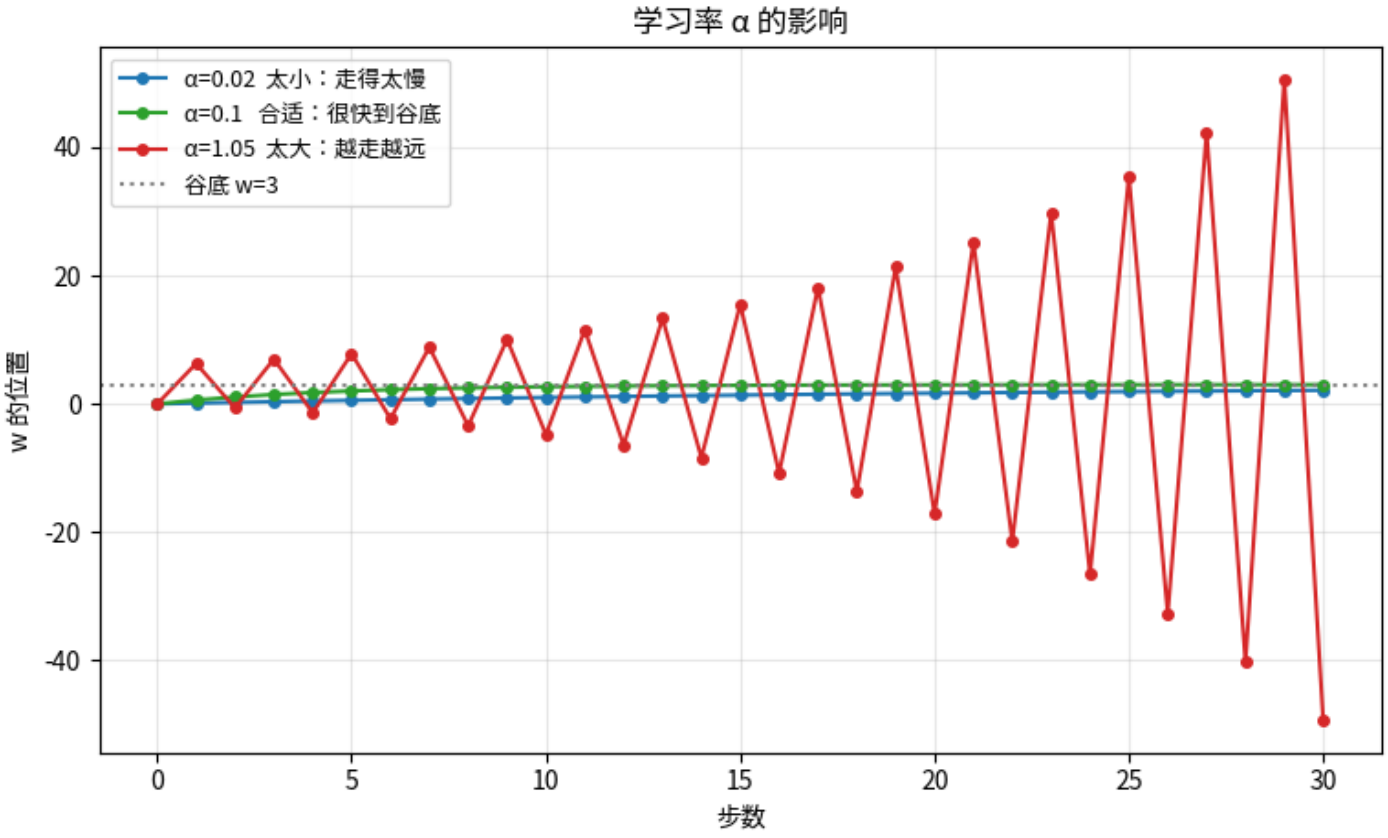
而且有个很美的性质：**越接近谷底，斜率越小，步子自动变小**——所以合适的 α 不会"冲过头"，会稳稳停在谷底。

跑代码看看三种 α 的区别：

In [4]:

```python
def descend(alpha, steps=30, start=0.0):
    w = start; hist = [w]
    for _ in range(steps):
        w = w - alpha * dJ(w)
        hist.append(w)
    return np.array(hist)

plt.figure(figsize=(7.8, 4.8))
for alpha, col, name in [(0.02, "tab:blue",  "α=0.02  太小：走得太慢"),
                         (0.1,  "tab:green", "α=0.1   合适：很快到谷底"),
                         (1.05, "tab:red",   "α=1.05  太大：越走越远")]:
    h = descend(alpha)
    plt.plot(range(len(h)), h, "o-", color=col, markersize=4, label=name)
plt.axhline(3, color="gray", linestyle=":", label="谷底 w=3")
plt.xlabel("步数"); plt.ylabel("w 的位置")
plt.title("学习率 α 的影响")
plt.legend(fontsize=9); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



看：绿线（α=0.1）几步就贴到谷底；蓝线（太小）磨磨蹭蹭；红线（太大）直接冲出画面飞走了。

> 记住：**α 不是越小越好，要"刚刚好"。** 真实训练里，选 α 是很重要的一门手艺。

五、两个参数：同时更新 w 和 b

真实模型有 `w` 和 `b` **两个旋钮**，所以要"分别求导、一起更新"：

$$w := w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$b := b - \alpha \frac{\partial}{\partial b} J(w,b)$$

这里新符号 `∂`（读"偏"）意思是：**"只对其中一个变量求导，把另一个先当常数"**。

- `∂/∂w J(w,b)`：**假装 b 不动**，只看 J 随 w 怎么变；
- `∂/∂b J(w,b)`：**假装 w 不动**，只看 J 随 b 怎么变。

（这就是为什么它叫"梯度"——两个方向的斜率合起来，指向最陡的上坡；我们**朝反方向**走，就是下坡。）

一个必须注意的细节：两个更新要**同时进行**。要先用**旧的** w、b 算出两个导数，**再一起更新**；不能先更新 w 再用新的 w 去算 b。程序里要写成：

```
tmp_w = w - alpha * dJ_dw(w, b)   # 先用旧的 w,b 算
tmp_b = b - alpha * dJ_db(w, b)
w, b = tmp_w, tmp_b               # 再一起赋值
```

六、把梯度下降用到线性回归

好消息：**线性回归的成本函数 J(w, b) 是一个完美的"碗"**（第 4 课见过），碗只有一个底。所以在它上面做梯度下降，**一定能走到全局最优**——不用担心陷在什么"半山腰的小坑"里。

把第 4 课的 `J(w,b)` 对 w、b 求偏导，代入更新式，就得到**线性回归的梯度下降**。这套算法每走一步都要用上"**全部 m 个训练样本**"，所以也叫 **Batch（批量）梯度下降**。

（偏导的具体式子下一课细讲；这一课你只要抓住：**代入、同时更新、重复，直到收敛。**）

> **整门课的核心闭环**：模型 `f(x)=wx+b` → 成本 `J(w,b)` → 梯度下降 → 得到最好的 (w, b)。

七、小结

1. **梯度下降**：蒙眼下山——反复感受坡度、朝下坡迈一步，直到谷底。
2. **核心更新式**：`w := w − α · dJ/dw`。**减号**让它自动朝下坡走。
3. **学习率 α**：管步子大小；太小太慢，太大发散，要合适。接近谷底时步子自动变小。
4. **两个参数**：`∂` 是"只对一个变量求导"，w、b 要**同时更新**。
5. **线性回归**：J 是碗状，梯度下降必到全局最优。

八、反转课堂 · 课后作业

1. 用自己的话说：梯度下降里那句 `w := w − α · dJ/dw`，**为什么是"减号"**？如果改成加号会发生什么？
2. 如果学习率 α 设得**太大**，J 会怎么变？设得**太小**呢？
3. 判断对错："更新 w 和 b 时，可以先把 w 更新好，再用新的 w 去更新 b。" 为什么？

下一课：把梯度下降**真正用到线性回归**上（第 6 课），推导它的偏导、看它怎么一步步逼近最优的拟合直线。